

33. Životní cyklus softwaru (charakteristika etap a základních modelů)

33. Životní cyklus softwaru

SZZ okruh 33 (FIT BUT). Etapy vývoje SW a modely, které je uspořádávají.

Shrnutí

Etapy

1. **Analýza a specifikace požadavků** (funkcionální/nefunkcionální; akceptační testy), 2. **architektonický + podrobný návrh**, 3. **implementace a testování částí**, 4. **integrace a testování**, 5. **provoz a údržba** (~67 % nákladů).
- **Verifikace** (děláme SW správně? akceptační testy) × **validace** (děláme správný SW? odpovídá potřebám).
- Pozdní oprava chyb je 50–200× dražší □ důraz na analýzu a návrh.

Lineární / heavyweight modely

- Velké projekty, stabilní požadavky, byrokracie, role > člověk.
- **Vodopádový** (sekvenční, SW až na konci □ problém validace), **V-model** (důraz na verifikaci/validaci, plánování testů), **spirálový** (analýza rizik, prototypy), **inkrementální** (po ucelených částech), **RUP** (iterativní framework, UML, milníky).

Iterativní a agilní

- Agilní: malé týmy, **adaptivní** přístup, intenzivní zapojení zákazníka, zpětná vazba, méně byrokracie.
- **XP** (komunikace, jednoduchost, testování/TDD, párové programování), **Scrum** (Sprint ~30 dní, Product/Sprint Backlog, Scrum Master, daily standup), RAD, Crystal.

UML pro návrh viz [[okruhy/34-uml]]; OO návrh viz [[okruhy/40-objektova-orientace]].

Časté otázky u zkoušky

Na co se u tohoto okruhu typicky ptají. Plné odpovědi níže.

Etapy □ [[#Etapy]] - *Jaké jsou etapy?* □ Analýza požadavků □ návrh □ implementace □ integrace/testování □ provoz a údržba. - *Verifikace vs. validace?* □ Verifikace = stavíme SW správně (akceptační testy); validace = stavíme správný SW (potřeby uživatele).

Modely □ [[#Lineární / heavyweight modely]] - *Vodopád vs. V-model?* □ Vodopád sekvenční (problém pozdní validace); V-model přidává plánování/provádění testů (verifikace+validace). - *Spirálový model?* □ Iterace s důrazem na analýzu rizik a prototypování. - *Lineární vs. iterativní — kdy?* □ Lineární pro stabilní požadavky (velké projekty); iterativní/agilní pro měnící se požadavky.

Agilní □ [[#Iterativní a agilní]] - *Scrum*? □ Sprints (~30 dní), Product/Sprint Backlog, denní Scrum, Scrum Master; inkrement na konci sprintu. - *Heavyweight* vs. *agilní*? □ Prediktivní + dokumentace + role × adaptivní + komunikace + lidé.

Testování □ [[#Etapy]] - *Typy testů*? □ Unit □ integrační □ systémové □ akceptační (u zákazníka).

Plné znění (ke studiu)

Životní cyklus softwaru

Mechanismus uplatňovaný při návrhu softwaru. Využívá se při vývoji velkých a složitých projektů. Je tvořena z **pěti etap**.

1. **Analýza a specifikace požadavků** (asi 8%),
2. **Architektonický a podrobný návrh** (asi 7%),
3. **Implementace a testování částí** (asi 12%),
4. **Integrace a testování** (asi 6%),
5. **Provoz a údržba** (asi 67%).

Bodům 1 a 2 je zejména nutné věnovat velkou pozornost, protože pozdější oprava chyb je **velmi drahá** (50krát až 200krát dražší než její oprava při analýze a návrhu).

Při vývoji SW jsou spolupracují 3 strany:

- **Zákazník** - Objednal SW (může být i samotným uživatelem, většinou to tak ale není, SW objednává management, pracují s ním běžní zaměstnanci).
- **Dodavatel** - Vytváří SW (analytici, programátoři, testéři, management).
- **Uživatel** - Bude SW používat.

Analýza a specifikace požadavků

Snažíme se **přesně specifikovat**, co zákazník chce (ale neřešíme, jak toho dosáhnout). Zákazník často neví, co přesně chce, nebo chce něco, co vlastně nepotřebuje. Úkolem analytiků je se zákazníkem požadavky probrat (**interview, dotazník, studium dokumentů, pozorování při práci, analýza existujícího SW**) a co nejvíce se přiblížit tomu, co opravdu potřebují. Výstupem je samotná **specifikace**, dále výstupem může být **analýza rizik** nebo **studie vhodnosti**, ale především **akceptační testy**, které vycházejí z jeho požadavků a které zákazník provede při převzetí. Výstupem také může být diagram případů užití, který slouží jako most mezi programátory a zákazníkem. Pokud jsou splněny akceptační testy, je SW v pořádku (**je verifikován**). Otázkou je, jestli software po implementaci opravdu plní funkce, které zákazník potřeboval - **validace**. Následné změny jsou ale poté již nad rámec projektu a zákazník je musí zaplatit jako rozšíření navíc.

Kategorie požadavků:

- **Funkcionální** - co má SW dělat.
 - definuje funkcionalitu systému, případně jeho komponent
 - popis vstup/chování/výstup
- **Nefunkcionální** – Co má dále o SW platit mimo jeho chování?
 - výkonnostní požadavky
 - bezpečnost
 - dostupnost (průměrný čas mezi výpadky)

- stabilita
- kapacita (zdroje aktuální, budoucí)
- **Požadavky na provoz systému** - za jakých podmínek bude systém pracovat (např. bude jej současně obsluhovat 200 lidí).
- **Požadavky na výsledný systém** - podmínky pro vývoj a nasazení.
- **Požadavky na vývojový proces** - požadavky zákazníka na dodržování norem (např. použití nějakého standardizované postupu autentizace).
- **Požadavky na rozhraní** - komunikace se systémem (např. webová aplikace).
- **Externí požadavky** - požadavky dle charakteru aplikace (např. legislativní požadavky).

Architektonický a podrobný návrh

Architektonický návrh se spíše zaměřuje na to, co budeme dělat; podrobný návrh na to, jak to budeme dělat.

Architektonický návrh Řeší se návrh aplikace od **fyzické úrovně** (např. že se bude jednat o třívrstvý informační systém s fyzicky odděleným DB serverem a klientem jako webový prohlížeč, i když část z toho může už být součástí specifikace - zákazník požaduje webovou aplikaci). Dále se může řešit jaké budeme používat nástroje pro vývoj - **programovací jazyky**. Následuje **logické členění aplikace**, např. výběr vhodného návrhového (architektonického) vzoru - **MVC, MVVM, MVP**. Následuje **rozdělení celého projektu na podproblémy** nejlépe nějak **logicky ucelené a rozhraní mezi nimi**. Můžeme je nazývat **moduly** (v případě MVC se může jednat o kontrolery). Příkladem může být modul pro správu uživatelů, ten bude skoro ve všech aplikacích. Členění do modulů může být výhodné také v tom, že na každém může pracovat jiný programátor - skupina programátorů. Nakonec by měly být produktem architektonického návrhu **návrh integračních testů a testů celého systému a plán nasazení do provozu**.

Podrobný návrh U informačního systému bude podrobný návrh spíše obnášet **návrh uložení dat v databázi** (např. pomocí ER diagramu) a **návrh uživatelského rozhraní**. U vědecké aplikace to bude spíše **návrh algoritmů a datových struktur**. Dále je součástí podrobného návrhu také návrh způsobu **ošetřování neočekávaných a chybových stavů**. Výstupem by měl být také **podrobný odhad ceny a nároky na lidské zdroje**. Výstupem také bude **návrh testů jednotlivých modulů**.

Implementace a testování částí

Nejedná se pouze o psaní kódu. V rámci implementace se také řeší **dokumentace** kódu, tvorba **uživatelské příručky/manuálu**. Chystá se **testovací prostředí**, které bude simulovat prostředí zákazníka a na kterém se bude aplikace testovat. Testovací prostředí je spojené s **tvorbou skriptů pro automatickou integraci (deployment) CI/CD**. **Testování** jednotlivých modulů a další.

Integrace a testování

Integrace je spojená se **začleňováním modulů** do jednoho celku - systému. Testování obnáší **integrační testy a testy systému jako celku**. Často se při provádění testů vracíme k předešlým dvěma bodům a **opravujeme vzniklé chyby**. Jakmile jsme přesvědčeni, že SW odpovídá specifikaci, předáváme jej zákazníkovi k akceptačnímu testování. To opět může odhalit nějaké chyby. Jakmile je ale akceptační testování provedeno a SW je tímto předán zákazníkovi, jsou následující změny již prováděny v rámci provozu a údržby.

Provoz a údržba

V první řadě se jedná o **opravu chyb**, které nebyly identifikovány ani akceptačním testováním. Dále jde o **aktualizaci použitých nástrojů** (např. z důvodu bezpečnostních rizik) a správu serveru, na kterém aplikace běží (někdy může provádět sám zákazník). Obecné řešení problémů při provozu, např. přesun na jiný server atd. Dále může jít o **změny** nebo o **přidání rozšiřujících funkcí** (postupně se přichází na to, že SW není validní - uživatelé doopravdy potřebují něco jiného, nebo změny mohou být způsobené např. legislativou). Nakonec je potřeba pravidelně provádět **zálohy dat, kontrolovat stav úložiště, kontrolovat vytížení serveru, ...**

Pojmy

Přehled pojmů, které asi úplně nejsou součástí okruhu, ale mohou se hodit.

Důležité vlastnosti u vývoje SW

- Splnění požadavků
- Cena
- Čas

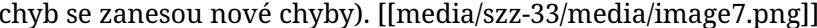
Typy SW produktu

- **Generický software** - SW **pro širokou veřejnost**, tzv. krabicový SW. Musí být velice dobře otestovaný (Ubuntu, Photoshop, Microsoft Office...).
- **Zákaznický software** - Vytvářen **pro určitého zákazníka**. Většinou pro danou specializovanou oblast neexistuje generický SW. Cena je mnohem větší (menší poptávka). Firma si ho přímo objedná (na zakázku) a specifikuje požadavky (IS VUT, FIT KIT).

Vlastnosti software

- **Použití** - Správnost, spolehlivost, efektivnost, použitelnost, bezpečnost.
- **Přenos** - Přenositelnost, znovupoužitelnost, interoperabilita (spolupráce s jinými systémy).
- **Změny** - Udržovatelnost, modifikovatelnost, testovatelnost, dokumentovanost.

Problémy při vývoji software

- **Nevyhnutelné** - Složitost, přizpůsobivost (na změny zadání), nestálost, neviditelnost (nevíme přesně, jak jsme blízko konci).
- **Ostatní** - Specifikace požadavků (nejasnosti v zadání), náchylnost k chybám (chyby se těžko odhalují), práce v týmu, nízká znovupoužitelnost, dokumentování, stárnutí softwaru (opravou chyb se zanesou nové chyby). 

Regresní testování (Regression testing)

Je provedení funkčních a nefunkčních testů pro zjištění, zda dříve vyvinutý a otestovaný software po změně stále funguje. Pokud se chyby objeví, říkáme, že došlo k regresi. Náprava chyb nalezených při regresním testování může zahrnovat opravy softwarových chyb, vylepšování softwaru, změny konfigurace, a dokonce i náhradu elektronických součástek. Sady regresních testů se obvykle zvětšují s každou nalezenou závadou, takže bývají velmi rozsáhlé, a proto se pro ně často využívá automatizace testů.

Modely životního cyklu SW

U každého projektu **se etapy mohou lišit**, společné rysy je však možné popsat **modely životního cyklu SW**. Definují etapy a jak po sobě jdou (ale ne však délku ani rozsah). Modely životního cyklu dělíme na **lineární** (Heavyweight a Agilní) a **iterační**.

Heavyweight modely

[[media/szz-33/media/image10.png]]

Využívají se většinou u **velkých projektů s velkým počtem programátorů, s centralizovaným řízením a prediktivním přístupem** (velké množství detailního plánování pro dlouhý časový úsek). **Malá flexibilita**, špatná reakce na změny uprostřed vývoje - vyžaduje **stabilní požadavky**. Funkcionalita je daná, potřebný čas a peníze je možné měnit. Malé nebo střední zapojení zákazníka při vývoji. Důraz je kladen na **procesy**, ty musí **běžet neustále** i při výměně zaměstnanců. Zaměstnanci jsou pouze zdroje dostupné v několika rolích (programátor, analytik, tester, ...) a lze je nahradit - **podstatná je role**. Předávání informací je založené především na dokumentaci (aby bylo možné obměňovat zaměstnance, u velkých projektů přes několik let, to ale jinak nemusí vždy jít) - **velká míra byrokracie**. Např. projekty NASA.

Vodopádový model

Etapy jsou řazeny za sebou **sekvenčně**, po skončení jedné začíná další. V případě nalezení chyb je potřeba se vrátit a projít znova. Neodpovídá reálnému vývoji, dnes už se jedná spíše o teoretický model.

- **výhody**: jednoduchý na řízení - srozumitelný, dobrá výsledná struktura - vše se dělá na poprvé a většinou se nepředělává/nevylepšuje.
- **nevýhody**: k zákazníkovi se SW dostává až na úplném konci vývoje, **problém s validací** a detekováním chyb. Následné opravy mohou být velmi drahé.

V-model

[[media/szz-33/media/image2.png]]

[[media/szz-33/media/image6.png]]

Vychází z vodopádového modelu, má stejné základní vlastnosti a výhody (zachovává jednoduchost a srozumitelnost vodopádového modelu), snaží se řešit nevýhody, zejména **validaci**. Písmeno V symbolizuje grafické uspořádání etap, ale také zdůrazňuje **validaci** a **verifikaci**. Dělí se levou a pravou část

- **levá část**: vývojové aktivity a plánování testů,
- **pravá část**: testovací aktivity - provádění testů. [[media/szz-33/media/image1.png]]

W-model

Vychází z V-modelu. Druhé V je spojeno s ověřováním jednotlivých aktivit (analýza, architektonický návrh, ...) a s testováním. Opět je model poměrně jednoduchý, ale k zákazníkovi se výsledný SW poprvé dostává až na konci vývoje, problém s validací přetrvává.

- **levá strana**:
 - **V1**: analýza, specifikace, architektonický návrh, podrobný návrh, implementace,
 - **V2**: ověřování výstupů etap z V1, plánování a návrh testů.

- pravá strana:
 - **V1:** provádění navržených testů,
 - **V2:** ladění, oprava chyb, změny v kódu, regresní testování. [[media/szz-33/media/image5.png]]

Iterativní (iterativní vodopádový) model

Proces je rozdělen do **iterací**, kde každá je **instance vodopádového modelu**. Výsledkem každé iterace je reálný výsledek (rozšíření systému/aplikace o novou funkci, oprava chyb z předešlé iterace).

- **Výhody:** tvorba reálných výsledků v každé iteraci umožňuje zákazníkovi výsledek validovat a lze rychleji odhalit chyby ve specifikaci, které mohou být napraveny v další iteraci.
- **Nevýhody:**
 - náročnější na řízení - vyžaduje účast zákazníka pro validaci po každé iteraci, že zákazník dostává výsledek, který si objednal. (Účast uživatele je až na výjimky, kdy je zákazník sám uživatelem, nemožná: Pokud vyvíjíme SW například pro sociální síť, nemůžeme v rámci vývoje zapojit uživatele (tedy uživatele sociální sítě), dokud SW není hotový. Nejde o finální nasazení, které by se dostalo k uživatelům sociální sítě, která ještě ani není nasazena.)
 - vede na horší strukturu kódu, z důvodu postupného přidávání funkcí a vylepšení. Lze však eliminovat **refaktorizací** (změna kódu bez změny jeho funkce - zákazník ale nebude chtít platit za refaktorizaci, „Když to nic nezmění?“ Vhodnou refaktorizací se ale může program zrychlit.).

[[media/szz-33/media/image11.png]]

Inkrementální model

Jedná se o kombinaci sekvenčních a iteračních metodik softwarového vývoje. Na základě specifikace se stanoví **ucelené části systému**, např. **moduly**, které jsou zákazníkovi postupně předávány (možnost validace a oprav). Následně se pracuje jedním ze dvou způsobů:

- pro **každou** (malou) část systému je prováděna **série vodopádů** (iterativní přístup), po jejím dokončení je tato **část systému předvedena zákazníkovi** a až poté se **začíná s další**.
- počáteční analýza, specifikace a návrh jsou provedeny **jedním vodopádem**. Vývoj (implementace) probíhá **iterativně** kombinovaně s **prototypováním**. V každé iteraci je systém rozšířen (vytvořen nový prototyp) a ten je prezentován zákazníkovi, nakonec je poslední prototyp považován za konečný systém.

Zhodnocení:

- **Výhody:** Inkrementální způsob umožňuje postupnou validaci a omezuje projektová rizika.
- **Nevýhody:** Vyžaduje zvýšenou pozornost při návrhu a implementaci rozhraní mezi moduly. Vývoj po částech může vést ke ztrátě vnímání logiky projektu jako celku.

Spirálový model

Zavádí do vývoje prototypování a klade **důraz na analýzu rizik**. Jednotlivé etapy se opakují vždy na **vyšším stupni zvládnuté problematiky** (analýza, specifikace, arch. návrh, ...), výsledkem je **prototyp**. Spirála je rozdělena do 4 kvadrantů:

1. **Stanovení cílů:** určení **funkčních a výkonnostních požadavků**, určení omezujících podmínek (čas a cena), návrh možných alternativ.
2. **Vyhodnocení stanovených cílů:** ověření jejich splnitelnosti, **analýza rizik, prototypování a simulace**,

3. **Realizace** stanovených cílů a jejich testování,
4. **Plánování** následujícího cyklu.

Význam jednotlivých cyklů (jejich počet není pevně stanovený), po každém cyklu je dokončený **Milestone**.

1. globální rizika, základní koncept vývoje, volba metod a nástrojů,
2. tvorba a ověřování specifikace požadavků,
 1. vyhodnocení záměrů a cílů projektu, rozhodnutí o dalším pokračování. Jsou identifikovány všechny požadavky a jejich chápání je shodné mezi zákazníkem a dodavatelem.
 2. Je vytyčena cena, plán priority aj.
 3. Jsou identifikovány rizika a postupy pro jejich zmírnění.
3. vytvoření a ověření návrhu,
 1. Vyhodnocení výběru architektury, požadavky a architektura jsou stabilní.
4. implementace, testování a integrace (vodopádový přístup)
 1. Je vytvořena stabilní verze schopná testování u zákazníka, pokud nejsou odhaleny problémy, je systém je připravený na distribuci pro uživatele.
 2. Porovnání plánovaných a skutečných výdajů.

Zhodnocení:

- Výhody:
 - Vhodný pro složité a velké projekty,
 - chyby a nevyhovující postupy jsou odhaleny dříve pomocí analýzy rizik.
- Nevýhody:
 - Analýza rizik musí být na vysoké úrovni, jinak postup může selhat
 - výsledný SW je k dispozici až po posledním cyklu (problém s validací). 

Analýza rizik

Zjištění možných ohrožení průběhu projektu a připravení reakce na tato ohrožení, lze včas vyloučit nevhodná řešení.

- **projektová:** odchod lidí, snížení rozpočtu,
- **technická:** neznámé technologie, selhání HW,
- **obchodní:** špatný odhad zájmů.

Rational Unified Process (RUP)

Nejedná se o model životního cyklu SW ani o konkrétní metodiku vývoje SW. Jde o **rozšiřitelný framework**, který je možné **upravit organizaci a příslušnému projektu**. Jedná se o **komerční produkt** (firma Rational Software), který je dodáváný společně s patřičnými nástroji. Je založený na osvědčených praktikách:

- **iterativní vývoj**, délka iterace je 2 až 6 týdnů,
- **vizualizovaný návrh systému** - využití UML a jiných nástrojů,
- **průběžná kontrola kvality**,

- řízení změn,
- využití existujících komponent.

Vývoj je rozdělen do několika cyklů. První nejdůležitější je **Initial Development Cycle** jehož výsledkem je funkční SW. Následný další vývoj (vylepšení, opravy) probíhá formou **Evolution Cycles**. Základní cyklus (Initial Development Cycle) je rozdělen na 4 fáze:

1. **Zahájení** (asi 10%) 1 až 2 iterace,
2. **projektování** (asi 30%) 2 až 4 iterace,
3. **realizace** (asi 50%) 2 až 4 iterace,
4. **předání** (asi 10%) aspoň 2 iterace (beta verze, plná verze).

Každá fáze je rozdělena na iterace o délce 2 až 6 týdnů (počet iterací se může lišit).

Ze spirálového modelu přebírá mezníky/milníky - **Milestones**. [[media/szz-33/media/image12.png]]

Zhodnocení:

- Výhody:
 - robustní, lze upravit pro potřeby projektu,
 - iterativní přístup, včasné odhalení rizik,
 - grafický návrh - vazba na UML,
 - detailní zpracovanost
- Nevýhody:
 - u malých projektů může představovat příliš velkou zátěž - neefektivní vývoj.
 - komerční produkt

Agilní metodiky

Využívají se většinou u **malých projektů s malým počtem programátorů, s decentralizovaným řízením** (vedení založené na spolupráci) a **adaptivním přístupem** (menší množství plánování, úpravy dle reakcí zákazníka) - **velká flexibilita**. Čas a peníze jsou obvykle dané a funkcionalita se implementuje na základě toho (co se stihne, co bude zapláceno; na co budou peníze, to bude naprogramováno). Vyžadují **intenzivní zapojení zákazníka** do procesu vývoje. Agilní metodiky kladou důraz na lidi a jejich **individualitu** (people-oriented), člověk je v dané oblasti profesionál (analytik, programátor, tester) a je schopen rozhodovat technické otázky práce, velmi důležitá je **kommunikace v rámci týmu**. **Omezuje se byrokracie** a formální požadavky. Ověření správnosti je prováděno **zpětnou vazbou** od zákazníka. Odchod schopných zaměstnanců může být kritický. Agilní metodiky dobře reagují na změny v průběhu vývoje, což je běžné.

Rapid Application Development

Metodika založená na **rychlém iterativním vývoji prototypů**. **Brzká** dispozice funkčních verzí, vyžaduje **intenzivní zapojení zákazníka** do vývojového procesu - **zpětná vazba** na poskytnuté verze. Zaměřuje se zejména na **splnění potřeb a požadavků zákazníka** (business potřeb SW). Metodika je určena pro menší a středně velké týmy. Zhodnocení:

- **Výhody:** flexibilita a schopnost rychlé reakce na změny požadavků od zákazníka. Vyšší kvalita zpracování business potřeb (zákazník dostává doopravdy to, co chce). Více projektů splňuje termíny a ceny.

- **Nevýhody:** nižší kvalita návrhu a výsledného kódu způsobená změnami, což vede na problémy s udržitelností.

Extrémní programování (XP)

Populární agilní metoda, která je založena na **komunikaci** (v týmu a se zákazníkem, zákazník se prakticky stává členem týmu) a **iterativním vývoji**. SW není dodán zákazníkovi celý v určitý termín v budoucnosti, ale je dodáván způsobem, který odráží aktuální potřebu uživatelů - **do systému vložíme to, co potřebujeme, když to potřebujeme**. Dva významní členové týmu:

- **kouč:** zajišťuje komunikaci v týmu, pomáhá programátorům s technickými dovednostmi, komunikuje s velkým šéfem a vyšším managementem.
- **velký šéf:** provádí zásadní rozhodnutí, komunikuje s vyšším managementem.

Extrémní programování klade velký důraz na **spolupráci v týmu**, manažeři, vývojáři a zákazník jsou si rovni. Je založeno na 5 základních principů:

1. **komunikace:** programátoři, zákazníci a manažeři spolu musí komunikovat - využití technik, které komunikaci vyžadují (pair programming, code review, ...).
2. **jednoduchost:** jednoduché věci se realizují a upravují s menším počtem chyb (v případě potřeby je lze rozšířit). Přírůstkové malé změny, uvolňování malých verzí systému (nejpodstatnější požadavky, které jsou postupně vylepšované a doplňované).
3. **zpětná vazba a testování:** vše musí být otestováno, ke každé funkci píšeme testy, klidně i před tím, než začneme programovat (Test Driven Development). Jednotkové i integrační testy, zpětná vazba zákazníka.
4. **odvaha:** nebát se zahodit naprogramovaný kód, nebát se zkusit neznámé. Pokud je potřeba, nebát se provést zásadní změny.
5. **respekt:** Každý malý úspěch prohlubuje respekt k jedinečným příspěvkům každého člena týmu.

Zhodnocení:

- **Výhody:**
 - iterativní inkrementální proces,
 - ladění na základě zpětné vazby - zapojení uživatelů,
 - založený na testování,
 - průběžná integrace
- **Nevýhody:**
 - vyžaduje dodržování základních principů (neustálé psaní testů, párové programování, atd)
 - nedefinuje přesný postup.

Collective-Code-Ownership Každý člen týmu má **možnost a povinnost ovlivňovat kód** (přidání nových funkcí, odstranění chyb, refaktorizace), což snižuje riziko, že výpadek člena týmu výrazně zbrzdí práci a podporuje pocit odpovědnosti vývojáře za kvalitu kódu (kdokoliv to po něm bude číst). Vyžaduje **jednotný styl programování**.

Průběžná integrace Automatizované sestavování, automatizované spouštění napsaných testů, automatizovaná publikace na testovací prostředí testování.

Scrum

Agilní metoda odvozená ze hry rugby. Lze kombinovat s XP. Dělí se na tři základní fáze:

- **pre-game:** plánování a **architektonický návrh**, využívá se **Product Backlog**.
- **game:** vývoj, který probíhá v iteracích, iterace se nazývá **Sprint** a trvá okolo **30 dní**, výsledkem každého sprintu je funkční inkrement. Na začátku každého sprintu je setkání mezi vývojáři, zákazníkem, uživateli atd. a definují se cíle sprintu - **Sprint Backlog** (seznam úloh nutných pro dosažení cíle). Každý den v průběhu sprintu se provádí **Scrum Meeting** - 15 min setkání členů vývojového týmu. **Scrum Master** je vedoucí týmu. Na konci sprintu se provádí **Sprint Review** - vyhodnocení proběhlého sprintu.
- **post-game:** integrace výsledků jednotlivých sprintů, testování (integrační, celého systému, akceptační), dokumentace, školení uživatelů.

Zhodnocení:

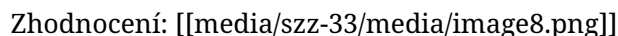
- **Výhody:**
 - iterativní inkrementální proces,
 - časté uvolňování verzí,
 - architektura je navržena před procesem vývoje,
 - jednoduchý proces,
 - zapojení uživatelů.
- **Nevýhody:**
 - nedefinuje přesný postup,
 - integrace až po vytvoření všech inkrementů.

Crystal

Jedná se o rodinu metologií, základní přístupy a techniky sdílí s **XP**. Více lidí je ale schopno tento proces akceptovat. Rozděluje projekty do **kategorií dle kritičnosti a důležitosti**:

- Comfort **C**,
- Discretionary Money (volné uvážení) **D**,
- Essential Money **E**,
- Life **L**.

Kategorie metodik, které se používají, se označují barvou, liší se dle velikosti týmu a délce projektu:

Zhodnocení: 

- **Výhody:**
 - iterativní inkrementální proces - časté uvolňování verzí,
 - průběžná integrace,
 - zapojení uživatelů - požadavky se ladí během celého vývoje na základě zpětné vazby
- **Nevýhody:**
 - nedefinuje jasný společný proces,
 - není vhodný pro vysoce kritické projekty,
 - velká závislost na přímé komunikaci.

Srovnání modelů

[[media/szz-33/media/image4.png]]

Zdroje

- SZZ okruh 33 — studijní materiály FIT BUT (szz-33.docx). Obrázky: media/szz-33/.

Navigace: [[index| • Přehled okruhů]] · □ [[okruhy/32-slozitost-algoritmu|32. Hodnocení složitosti algoritmů]] · další: [[okruhy/34-uml|34. Jazyk UML]] □